

SYSTEM SOFTWARE

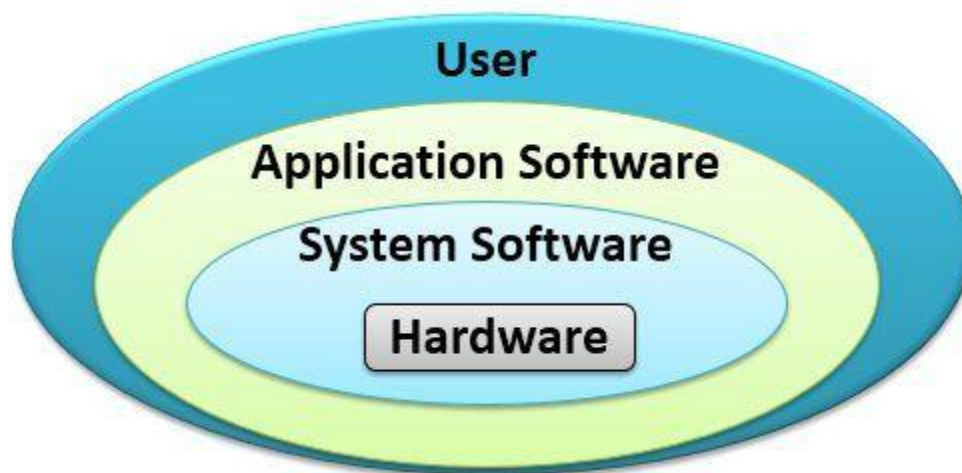
(UNIT-1)

Definition

System software is a set of programs designed to **control, manage, and operate the computer hardware** and provide a platform for running application software. It acts as an **interface between the user/application programs and computer hardware**.

Main Functions of System Software

1. **Hardware management** – Controls CPU, memory, I/O devices, storage, etc.
2. **Resource allocation** – Allocates system resources to different programs.
3. **Program execution** – Loads programs into memory and helps execute them.
4. **File management** – Organizes and manages files and directories.
5. **Error handling** – Detects and handles hardware and software errors.
6. **Provides an interface** – Provides an environment through which application programs can use hardware.



Characteristics

- Works closely with computer hardware.
- Usually runs in the background.
- Provides a platform for application software.
- Improves efficient utilization of system resources.

- Generally essential for the proper functioning of a computer.

System Software vs Application Software

Basis	System Software	Application Software
Definition	Software that manages and controls computer hardware and provides a platform for applications.	Software designed to perform specific tasks for the user.
Purpose	To operate and manage the computer system.	To help users perform particular tasks.
Dependency	Application software depends on system software to run.	Depends on system software for execution.
Interaction	Works mainly with hardware and system resources.	Works mainly with the user.
Execution	Often runs in the background.	Usually runs when the user requests it.
Examples	Operating systems, assemblers, compilers, loaders, device drivers.	MS Word, Excel, PowerPoint, web browsers, media players.
Necessity	Essential for the basic operation of a computer.	Not essential for basic computer operation; depends on user requirements.
Development	Generally more complex and hardware-dependent.	Generally developed to meet specific user requirements.

What is Machine Architecture?

Definition

Machine Architecture refers to the **design and organization of a computer system at the machine level**. It describes how the **CPU, memory, input/output devices, registers, instruction set, and other hardware components** are organized and how they work together to execute instructions.

In System Software, understanding machine architecture is important because **system software such as assemblers, compilers, and operating systems must interact closely with the underlying hardware.**

Main Components of Machine Architecture

1. **Central Processing Unit (CPU)**

The CPU executes instructions and performs arithmetic and logical operations.

2. **Registers**

Small, high-speed storage locations inside the CPU used to hold data, addresses, and intermediate results.

Examples: **Accumulator, Program Counter (PC), Instruction Register (IR), and General-Purpose Registers.**

3. **Memory**

Stores programs, instructions, and data required during program execution.

4. **Instruction Set**

A collection of machine instructions that the processor can understand and execute.

Examples include **LOAD, STORE, ADD, SUBTRACT, and JUMP.**

5. **Addressing Modes**

Specify how the operand or its address is obtained. Examples include **immediate, direct, indirect, indexed, and register addressing.**

6. **Input/Output (I/O) System**

Provides communication between the computer and external devices such as keyboards, displays, printers, and storage devices.

7. **Data and Instruction Formats**

Define how data and instructions are represented and stored in memory.

Importance of Machine Architecture

- Helps system software understand and control hardware.
- Determines how machine instructions are represented and executed.
- Defines the available registers and addressing methods.
- Determines how memory and I/O devices are accessed.
- Helps in designing assemblers, compilers, and operating systems.
- Influences the performance and efficiency of a computer system.

CPU

Definition

The **CPU (Central Processing Unit)** is the main processing unit of a computer. It is often called the “**brain of the computer**” because it controls the operations of the computer and executes instructions stored in memory.

The CPU performs **arithmetic, logical, control, and data-processing operations**.

Main Components of CPU

1. Arithmetic Logic Unit (ALU)

- Performs arithmetic operations such as **addition, subtraction, multiplication, and division**.
- Performs logical operations such as **AND, OR, NOT, and comparisons**.

2. Control Unit (CU)

- Controls and coordinates all activities of the computer.
- Fetches instructions from memory, decodes them, and generates control signals for their execution.

3. Registers

- Small and very fast storage locations inside the CPU.
- Temporarily store data, instructions, addresses, and intermediate results.
- Examples: **Accumulator (A), Program Counter (PC), Instruction Register (IR), and General-Purpose Registers**.

4. Cache Memory

- High-speed memory located inside or close to the CPU.
- Stores frequently used instructions and data to reduce access time.

Working of CPU

The CPU generally follows the **Instruction Cycle**:

1. Fetch → 2. Decode → 3. Execute → 4. Store

Functions of CPU

- Executes program instructions.
- Performs arithmetic and logical operations.
- Controls the activities of other computer components.

- Processes data and produces results.
- Transfers data between registers, memory, and I/O devices.
- Maintains the sequence of instruction execution.

MACHINE LANGUAGE

Definition

Machine Language is the **lowest-level programming language** that is directly understood and executed by the computer's CPU. It consists of instructions written in **binary digits (0s and 1s)**.

Each machine-language instruction corresponds to a specific operation that the processor can perform.

Advantages

- **Very fast execution** because instructions are directly understood by the CPU.
- **Efficient use of hardware resources.**
- Gives programmers **direct control over hardware.**
- Does not require a compiler or assembler for execution.

Disadvantages

- Very difficult to write and understand.
- Difficult to debug and modify.
- Machine-dependent and not portable.
- Programming large applications is extremely time-consuming.
- Binary code is highly prone to errors.

Machine Language

Uses **0s and 1s**

Directly understood by CPU

Very difficult for humans

Machine dependent

Assembly Language

Uses **mnemonics and symbols**

Requires an **assembler**

Easier for humans

Machine dependent

Machine Language

Example: 10110000

Assembly Language

Example: MOV A, B

SIMPLIFIED INSTRUCTIONAL COMPUTER (SIC) – MACHINE ARCHITECTURE

The **Simplified Instructional Computer (SIC)** is a hypothetical computer system designed by **Leland L. Beck** to explain the fundamental concepts of **system software**, especially **assemblers, loaders, and compilers**.

SIC provides a simple machine architecture while containing the basic features found in real computers.

Main Features of SIC

- **Memory:** 32,768 bytes (32 KB).
- **Word size:** 24 bits (3 bytes).
- **Registers:** Five main registers.
- **Instruction length:** 3 bytes (24 bits).
- **Address space:** 15-bit addresses.
- Uses **two's complement** representation for signed integers.
- Supports **character data** using ASCII codes.
- Provides **input/output instructions** for devices.

SIC Registers

SIC has **five registers**, each having a length of 24 bits.

Register Name		Function
A	Accumulator	Used for arithmetic and logical operations
X	Index Register	Used for indexed addressing
L	Linkage Register	Stores the return address during subroutine calls
PC	Program Counter	Contains the address of the next instruction

Register Name	Function
SW Status Word	Contains condition code and other status information

Important Registers

- **A (Accumulator):** Stores operands and results of arithmetic operations.
- **X (Index Register):** Used to modify addresses for indexed addressing.
- **L (Linkage Register):** Stores the return address when a subroutine is called.
- **PC (Program Counter):** Keeps track of the next instruction to be executed.
- **SW (Status Word):** Stores the condition code such as <, =, or >.

Addressing Modes in SIC

Addressing mode specifies **how the address of an operand is determined** from the instruction.

In **SIC**, there are mainly **two addressing modes**:

1. Direct Addressing Mode

- This is the **default addressing mode** in SIC.
- The **x bit is set to 0**.
- The address field of the instruction directly contains the address of the operand.
- The effective address is:

EA = Address

- No modification is made using the X register.
- It is simple and commonly used.

2. Indexed Addressing Mode

- Indexed addressing is indicated when the **x bit is set to 1**.
- The contents of the **X (Index) register** are added to the address field.
- The effective address is:

EA = Address + X

- It is useful for accessing **arrays, tables, and sequential data**.
- By changing the value of X, different elements of an array can be accessed.

Important Points to Remember

1. SIC instruction contains an **address field of 15 bits**.
2. The **x bit** determines whether indexed addressing is used.
3. **x = 0 → Direct addressing**
4. **x = 1 → Indexed addressing**
5. The **X register** is used only for indexed addressing.
6. In direct addressing, **EA = Address**.
7. In indexed addressing, **EA = Address + X**.
8. Indexed addressing is particularly useful for **arrays and tables**.
9. SIC does **not** have separate immediate or register addressing modes like SIC/XE.
10. The addressing mode is determined directly from the **instruction format**.

SIC Instruction Format

Each SIC instruction is **3 bytes (24 bits)** long.

Opcode	x	Address	
---- 8 bits ----	--- 1 bit ---	----- 15 bits -----	

- **Opcode (8 bits):** Specifies the operation to be performed.
- **x (1 bit):** Indicates whether indexed addressing is used.
- **Address (15 bits):** Specifies the memory address of the operand.

SIC Instruction Set

An **instruction set** is the collection of instructions that a computer can understand and execute.

SIC instructions can be grouped into several categories.

A. Data Transfer Instructions

Used to move data between registers and memory.

Instruction Meaning

LDA	Load A
LDX	Load X
LDL	Load L

Instruction Meaning

STA Store A

STX Store X

STL Store L

B. Arithmetic Instructions

Used for mathematical operations.

Instruction Meaning

ADD Addition

SUB Subtraction

MUL Multiplication

DIV Division

C. Comparison Instruction

COMP

Compares the value in accumulator A with a value in memory.

COMP NUM

The result is reflected in the **Status Word (SW)**.

It can indicate:

- A < NUM
- A = NUM
- A > NUM

D. Branching Instructions

Used to change the normal sequence of program execution.

Instruction Meaning

J Jump

JEQ Jump if equal

JLT Jump if less than

Instruction Meaning

JGT	Jump if greater than
JSUB	Jump to subroutine
RSUB	Return from subroutine

5. I/O in SIC

I/O = Input/Output

SIC provides instructions to communicate with external devices such as keyboards, printers, etc.

There are **three main I/O instructions**.

1. TD – Test Device

Checks whether an I/O device is ready.

TD DEVICE

2. RD – Read

Reads a byte from an input device.

RD DEVICE

The data is generally placed in the **least significant byte of register A**.

3. WD – Write

Writes a byte to an output device.

WD DEVICE

SIC/XE MACHINE ARCHITECTURE

Introduction

SIC/XE (Simplified Instructional Computer / Extra Equipment) is an **extended version of the SIC architecture**. It was designed to provide more powerful features while still remaining simple enough to study **system software concepts**, especially assemblers and loaders.

SIC/XE provides:

- More registers
- Larger memory

- More instruction formats
- More addressing modes
- Floating-point operations
- Better I/O facilities

Main Features of SIC/XE

1. **Memory size:** Up to 1 MB (2^{20} bytes).
2. **Word size:** 24 bits (3 bytes).
3. **Registers:** 9 registers.
4. Supports **floating-point arithmetic**.
5. Supports **four instruction formats**: Format 1, 2, 3 and 4.
6. Supports several addressing modes such as **immediate, indirect, simple, and indexed addressing**.
7. Supports **PC-relative and base-relative addressing**.
8. Provides **program-counter (PC) relative addressing**.
9. Provides **base-relative addressing**.
10. Supports **extended format instructions** for larger addresses.
11. Provides improved **I/O instructions**.
12. Supports **supervisor-call instructions**.

SIC/XE Registers

Register	Size	Purpose
A	24 bits	Accumulator
X	24 bits	Index register
L	24 bits	Linkage register
B	24 bits	Base register
S	24 bits	General-purpose register
T	24 bits	General-purpose register
F	48 bits	Floating-point register

Register	Size	Purpose
----------	------	---------

PC	24 bits	Program Counter
----	---------	-----------------

SW	24 bits	Status Word
----	---------	-------------

PURPOSE

- **A:** Used for arithmetic and logical operations.
- **X:** Used for indexed addressing.
- **L:** Stores the return address for subroutines.
- **B:** Used for base-relative addressing.
- **S and T:** General-purpose registers.
- **F:** Used for floating-point operations.
- **PC:** Contains the address of the next instruction.
- **SW:** Contains status and condition information.

SIC/XE Instruction Formats

SIC/XE has **four instruction formats**.

Format 1

- Length: **1 byte**
- Contains only the opcode.

Format 2

- Length: **2 bytes**
- Used mainly for register-to-register operations.
- Contains an opcode and two register numbers.

Format 3

- Length: **3 bytes**
- Used for most memory-reference instructions.
- Contains opcode, addressing flags, and displacement.

Format 4

- Length: **4 bytes**
- Similar to Format 3 but provides a larger address field.
- The **e bit = 1**.
- Used when the address cannot be represented using Format 3 displacement.

Addressing Modes in SIC/XE

SIC/XE supports several addressing modes.

1. Immediate Addressing

- **i = 1, n = 0**
- Operand value is given directly in the instruction.
- Example:

LDA #5

Here, **5 is the actual value**, not a memory address.

2. Indirect Addressing

- **i = 0, n = 1**
- The address field points to a memory location that contains the address of the operand.

Example:

LDA @ALPHA

3. Simple Addressing

- **n = 1, i = 1**
- The address field refers to the operand's memory location.

Example:

LDA ALPHA

4. Indexed Addressing

- **x = 1**
- The contents of X are added to the calculated address.

$$EA = Address + X$$

It is useful for accessing **arrays and tables**.

5. PC-Relative Addressing

- **p = 1**
- Effective address is calculated using the Program Counter.

$$EA = (PC) + displacement$$

It allows programs to be **relocatable**.

6. Base-Relative Addressing

- **b = 1**
- Effective address is calculated using the Base register.

$$EA = (B) + displacement$$

It is useful when the target address is outside the PC-relative range.

Instruction Format Flags

Format 3 and Format 4 use several important flags:

Flag	Meaning	Function
n	Indirect bit	Specifies indirect/simple addressing
i	Immediate bit	Specifies immediate/simple addressing
x	Index bit	Enables indexed addressing
b	Base bit	Enables base-relative addressing
p	PC bit	Enables PC-relative addressing
e	Extended bit	Selects Format 4

Important combinations

n i Addressing

0 1 Immediate

1 0 Indirect

1 1 Simple

n i Addressing

0 0 SIC compatibility

SIC vs SIC/XE

Basis	SIC	SIC/XE
1. Full Form	Simplified Instructional Computer	Simplified Instructional Computer / Extra Equipment
2. Memory Size	32 KB (32,768 bytes)	Up to 1 MB (2²⁰ bytes)
3. Registers	5 registers: A, X, L, PC, SW	9 registers: A, X, L, B, S, T, F, PC, SW
4. Register Size	24 bits	24 bits
5. Instruction Formats	Mainly 3-byte format	4 formats: 1, 2, 3 and 4
6. Addressing Modes	Mainly direct and indexed	Immediate, indirect, simple, indexed, PC-relative, base-relative
7. Floating Point	Not supported	Supported through F register
8. Instruction Length	Fixed 3 bytes	1, 2, 3, or 4 bytes
9. Addressing Range	15-bit address field	Supports 20-bit addresses and larger addressing through Format 4
10. Register Operations	Limited register operations	Supports more register-to-register operations using Format 2
11. Base Register	No dedicated base register	B register is provided for base-relative addressing
12. Immediate Addressing	Not available	Available, e.g., LDA #5
13. Indirect Addressing	Not available	Available, e.g., LDA @ALPHA

Basis	SIC	SIC/XE
14. PC-Relative Addressing	Not supported	Supported
15. Extended Format	Not available	Format 4 provides extended addressing
16. I/O Facilities	Basic I/O instructions	More advanced I/O facilities
17. Complexity	Simpler	More powerful and complex
18. Purpose	Used to understand basic machine architecture	Used to study more advanced system software concepts

Machine Structure

Machine structure describes **how the major hardware components of a computer are organized and how they communicate with each other**. The two most important components are **memory** and the **processor (CPU)**.

1. Memory

Memory is the part of a computer used to store **data, instructions, and results**.

Types of Memory

A. Primary Memory

- Directly accessible by the CPU.
- Faster than secondary storage.
- Mainly includes:
 - **RAM (Random Access Memory):** Temporary/volatile storage. Data is lost when power is turned off.
 - **ROM (Read Only Memory):** Non-volatile memory that stores permanent instructions.

B. Secondary Memory

- Used for long-term storage.
- Larger capacity but slower than primary memory.

- Examples: **Hard disk, SSD, pen drive, memory card.**

Memory Hierarchy

Memory is arranged according to **speed, cost, and capacity**:

Registers → Cache → Main Memory (RAM) → Secondary Storage

- **Registers:** Fastest and smallest.
- **Cache:** Very fast memory between CPU and RAM.
- **RAM:** Main working memory.
- **Secondary storage:** Slowest but has the largest capacity.

2. Processor (CPU)

The **processor**, or **Central Processing Unit (CPU)**, is the main component responsible for **executing instructions and processing data**.

The CPU mainly consists of:

A. ALU (Arithmetic Logic Unit)

Performs:

- Arithmetic operations: addition, subtraction, multiplication, division.
- Logical operations: AND, OR, NOT, comparison, etc.

B. Control Unit (CU)

- Controls and coordinates all operations of the computer.
- Fetches instructions from memory.
- Decodes the instructions.
- Generates control signals for execution.

C. Registers

Small, high-speed storage locations inside the CPU.

Examples:

- **Program Counter (PC):** Holds the address of the next instruction.
- **Instruction Register (IR):** Holds the current instruction.
- **Accumulator (ACC):** Stores intermediate results.
- **General-purpose registers:** Temporarily store data and operands.

3. How Memory and Processor Work Together

The basic operation is the **Fetch–Decode–Execute cycle**:

Memory → CPU → Execute → Result

1. **Fetch**: CPU fetches an instruction from memory.
2. **Decode**: Control Unit understands what the instruction means.
3. **Execute**: ALU or another CPU component performs the operation.
4. **Store**: Result is stored in a register or memory.

CORE CPU COMPONENTS: MAR AND MBR

Two important registers used when the CPU communicates with **main memory** are the **Memory Address Register (MAR)** and **Memory Buffer Register (MBR)**.

1. Memory Address Register (MAR)

The **MAR** stores the **address of the memory location** that the CPU wants to access.

- It contains an **address**, not the actual data.
- When the CPU wants to read or write memory, it places the required memory address in the MAR.
- The MAR is connected to the **address bus**.

Example:

If the CPU wants to access memory location **2050**:

MAR ← 2050

This means: "Access memory location 2050."

2. Memory Buffer Register (MBR)

The **MBR** stores the **data or instruction being transferred between the CPU and memory**.

It is also commonly called the **Memory Data Register (MDR)**.

- During a **read operation**, data coming from memory is placed in the MBR.
- During a **write operation**, data to be stored in memory is placed in the MBR.
- It is connected to the **data bus**.

Example:

If memory location 2050 contains the value **75**:

$MBR \leftarrow \text{Memory}[2050]$

So, MBR now contains **75**.

How They Work Together

Suppose CPU wants to **read data from address 2050**:

Step 1: $MAR \leftarrow 2050$

Step 2: CPU sends read signal to memory.

Step 3: $MBR \leftarrow \text{Memory}[MAR]$

Step 4: Data in MBR is transferred to the CPU.

Basis	MAR	MBR
1. Full Form	Memory Address Register	Memory Buffer Register
2. Purpose	Holds the address of the memory location to be accessed.	Holds the data or instruction being transferred to/from memory.
3. Stores	Memory address	Data/instruction
4. Direction	Sends the address to memory.	Transfers data between CPU and memory.
5. Connected to	Address bus	Data bus
6. During Read	Contains the address from which data is read.	Receives the data read from memory.
7. During Write	Contains the address where data will be written.	Contains the data that will be written to memory.
8. Example	If CPU wants data from address 5000 , $MAR = 5000$.	If data at address 5000 is 25 , $MBR = 25$.
9. Function	Identifies where the operation takes place.	Holds what is being transferred.
10. Main Role	Handles address information .	Handles data/instruction information .

DATA FORMATS

Data formats specify how many bits are used to represent a value.

A. Long-Form Data — 32-bit

A **long-form** data item occupies **32 bits = 4 bytes**.

It can be represented as:

[32-bit data]

B. Short-Form Data — 16-bit

A **short-form** data item occupies **16 bits = 2 bytes**.

It can be represented as:

[16-bit data]

Basis	Short Form Data	Long Form Data
Size	Generally 32 bits (4 bytes)	Generally 64 bits (8 bytes)
Precision	Lower precision	Higher precision
Memory required	Less memory	More memory
Range	Smaller range	Larger range
Floating-point	Single-precision floating point	Double-precision floating point
Accuracy	Lower accuracy	Higher accuracy
Execution	Usually faster, fewer resources	May require more resources
Use	When normal precision is sufficient	When high precision is required
Example	Single-precision floating-point number	Double-precision floating-point number

INSTRUCTION FORMATS

These are **IBM System/360 instruction formats** commonly discussed in System Software and assembler topics. The letters indicate the types of operands/registers involved.

1. RR Format — Register to Register

RR = Register-Register

- Instruction length: **2 bytes (16 bits)**
- Contains an **8-bit opcode** and two **4-bit register fields**.
- Both operands are registers.
- Used for operations between two registers.

Format

| Opcode | R1 | R2 |

| 8 bits | 4bit | 4bit |

- **R1** = First register
- **R2** = Second register

2. RX Format — Register and Indexed Storage

RX = Register-Indexed

- Instruction length: **4 bytes (32 bits)**
- Used when one operand is a **register** and the other is a **memory location**.
- Supports **indexing** and **base-displacement addressing**.

Format

| Opcode | R1 | X2 | B2 | D2 |

| 8 bits | 4bit | 4bit | 4bit | 12 bits |

- **R1** = Register containing the first operand
- **X2** = Index register
- **B2** = Base register
- **D2** = Displacement

Effective Address

EA = Contents of B2 + Contents of X2 + D2

3. RS Format — Register and Storage

RS = Register-Storage

- Instruction length: **4 bytes (32 bits)**
- Contains **two register fields** and a **12-bit displacement**.

- One register may act as an operand and the other may specify a **register pair** or be unused depending on the instruction.

Format

Opcode	R1	R3	B2	D2
8 bits	4bit	4bit	4bit	12 bits

Effective Address

EA = Contents of B2 + D2

If indexing is applicable:

EA = Contents of B2 + Contents of X + D2

Important Point

RS format is especially useful for **multiple-register operations**.

4. SI Format — Storage Immediate

SI = Storage-Immediate

- Instruction length: **4 bytes (32 bits)**
- One operand is a **memory location**.
- The other operand is an **8-bit immediate value**.

Format

Opcode	I2	B1	D1
8 bits	8 bits	4bit	12 bits

- **I2** = Immediate value
- **B1** = Base register
- **D1** = Displacement

Effective Address

EA = Contents of B1 + D1

5. SS Format — Storage-to-Storage

SS = Storage-Storage

- Instruction length: **6 bytes (48 bits)**
- Both operands are **memory locations**.

- Contains two base-displacement address specifications.
- Some SS instructions also contain a **length field**.

Format

| Opcode | L1 | L2 | B1 | D1 | B2 | D2 |
 | 8 bits | 4 bits | 4 bits | 4bit | 12bit | 4bit | 12bit |

- **L1/L2** = Length fields
- **B1/B2** = Base registers
- **D1/D2** = Displacements

Effective Addresses

EA₁ = Contents of B1 + D1

"LONG WAY – NO LOOPING" METHOD

The **Long Way – No Looping method** is a simple way of writing an assembly language program where **each operation is written separately**, without using a loop or branch instruction to repeat the operation.

It is useful for understanding the basic working of **assembly instructions, registers, memory, and addressing modes**.

How it works

1. LDA NUM1 → Load NUM1 into the accumulator.
2. ADD NUM2 → Add NUM2 to the accumulator.
3. ADD NUM3 → Add NUM3.
4. ADD NUM4 → Add NUM4.
5. ADD NUM5 → Add NUM5.
6. STA RESULT → Store the final result in RESULT.

So:

RESULT = NUM1 + NUM2 + NUM3 + NUM4 + NUM5

Long Way – No Looping

Each operation is written separately

Program is longer

No branch/repetition

Easy for small number of items

More repetitive

Looping Method

Same operation is repeated using a loop

Program is shorter

Uses branch instructions

Better for large number of items

Less repetitive

Long Way – No Looping means performing each operation individually without using a loop. It is simple to understand but results in a longer and less efficient program when processing a large number of data items.

Assembly Language Basics

Assembly language is a **low-level programming language** that uses **mnemonics** to represent machine instructions.

There are two important types of operations:

1. **Machine OPs (Machine Operations)**
2. **Pseudo OPs (Pseudo Operations / Assembler Directives)**

1. Machine OPs (Machine Operations)

Machine OPs are assembly language instructions that are **translated into actual machine instructions** and are executed by the CPU.

They specify an operation that the computer must perform.

Characteristics

1. They represent **actual CPU operations**.
2. They are converted into **machine/object code** by the assembler.
3. They are **executed during program execution**.
4. They perform operations such as loading, storing, arithmetic, comparison, and branching.
5. They use CPU registers and memory.
6. They generally have corresponding **opcodes**.

7. They are dependent on the instruction set of the processor.
8. Examples include **L, A, ST, B, BALR, BR, C, AR**, etc.

2. Pseudo OPs (Pseudo Operations / Assembler Directives)

Pseudo OPs are statements that provide **instructions to the assembler**, rather than instructions to the CPU.

They **do not normally generate executable machine instructions**. Instead, they help the assembler organize the program, define data, allocate storage, define symbols, and control the assembly process.

Characteristics

1. They are **not executed by the CPU**.
2. They provide instructions to the **assembler**.
3. They generally do not have machine opcodes.
4. They help in **memory allocation and symbol definition**.
5. They control the **assembly process**.
6. They may generate data or reserve memory even though they do not generate executable instructions.
7. They make assembly programs easier to organize.
8. Examples include **START, END, DS, DC, USING, DROP, EQU, and LTORG**.

Basis	Machine OPs *****	Pseudo OPs *****
Meaning	Machine Operations	Pseudo Operations
Purpose	Tell CPU what operation to perform	Tell assembler what to do
Execution	Executed by CPU	Not executed by CPU
Machine code	Generate machine/object instructions	Generally do not generate executable instructions
Opcode	Have corresponding machine opcode	Usually have no CPU opcode
Use	Data processing, branching, loading, storing	Storage allocation, constants, symbols, assembly control
Examples	L , A , ST , BR , BALR	START , END , DS , DC , USING
Processor dependent	Yes	Mainly assembler dependent

